WEB DEVELOPMENT

WITH PYTHON

Copyright © 2025 Skill Foundry

INTRODUCTION 3
Setting Up Your Development Environment 6
Understanding HTTP, HTML & APIs 10
Introduction to Flask 16
Building a Mini Website with Flask 21
Introduction to Django (Fast Overview) 27
Creating a Simple Blog App with Django 36
8 Hosting Your Web App 40
Bonus Projects (Mini Real World Projects) 44

Copyright © 2025 Skill Foundry

INTRODUCTION

What is Web Development?

Imagine you open your favorite website — Instagram, YouTube, or Amazon.
The moment you type the URL and hit "Enter," magic happens behind the scenes.
That magic is called **Web Development**.

In simple terms:

Web Development means creating websites or web applications that you can access through the internet using a browser like Chrome, Firefox, or Safari.

Web development is mainly divided into two parts:

- **Frontend Development**:
  Whatever you see and interact with on a website — buttons, forms, text, images — is all frontend.
- **Backend Development**:
  Whatever happens behind the scenes — storing data, handling user login, sending emails — is all backend.

Python is mainly used for **backend development**, but you will learn how it connects with the frontend to build complete web apps!

Why Python for Web Development?

You might wonder —
"There are so many languages like PHP, Ruby, or Node.js... why should I use Python?"

Here's why Python is a champion choice:

- ✅ **Easy to Learn**:
  Python code looks like plain English, which makes it beginner-friendly.
- ✅ **Powerful Frameworks**:
  Python has super-powerful tools like **Flask** and **Django** that make building websites fast and easy.
- ✅ **Massive Community Support**:
  If you ever get stuck, you'll find thousands of tutorials, forums, and people ready to help.
- ✅ **Career Opportunities**:
  Python developers are highly in demand, especially those who can build web apps and APIs.

Copyright © 2025 Skill Foundry

- ✅ **Works Well with Other Skills**:
  After web development, you can combine Python with AI, Machine Learning, or Data Analytics too!

How Web Applications Work (Simplified View)

Let’s break it down into a simple example:

When you visit Amazon and search for "shoes," here’s what happens:

1. **Frontend**:
   You see a search box where you type "shoes" and click on the search button.
2. **Backend**:
   Python code (running on Amazon’s servers) reads your request, searches its database for "shoes," and prepares the matching results.
3. **Frontend again**:
   The results (list of shoes) are shown to you beautifully on the web page.

In short:
👉 *Frontend = What you see*
👉 *Backend (Python) = How it works internally*

Understanding Frontend vs Backend — With a Simple Analogy

Let’s say you go to a restaurant:

- You sit at a table and look at the **menu**. (Frontend)
- You call the **waiter** and place your order. (Frontend action)
- The waiter goes into the **kitchen** and tells the **chef** what you ordered. (Backend communication)
- The chef **prepares the food** inside the kitchen (Backend processing).
- The waiter **brings the food** to your table. (Frontend shows the result)

In this analogy:

Part	What it Represents
Menu & Table	Frontend (HTML, CSS, JavaScript)
Waiter	API (connects frontend to backend)
Chef & Kitchen	Backend (Python server + Database)

Copyright © 2025 Skill Foundry

Moral:
Frontend is what the user interacts with. Backend is where the real work happens.

Core Skills Needed for Web Development with Python

Here's a list of skills you'll be learning (and don't worry — you'll learn them easily through this guide):

Skill	Description
HTML Basics	Structure the webpage (the skeleton)
CSS Basics	Style the webpage (make it beautiful)
Python	Main language for backend
Flask/Django	Frameworks to create websites quickly
API Basics	Connect frontend and backend
Database Basics	Store user data (like signup info, product details)

Pro Tip :

You don't need to master frontend right now! Just basic HTML and CSS knowledge is enough to start backend development with Python.

How We Will Learn in This Guide

- 🔥 **Fast-track approach** (No boring long theories)
- 🛠️ **Hands-on mini projects** (Create real working apps)
- 🎯 **Goal-oriented structure** (Every chapter = small achievement)
- **Step-by-step building** (From simple to advanced)

This means you won't just "read" about Flask, Django, and APIs —
You will actually build working mini apps like:

- A mini blog
- A simple login/signup app
- An API for fetching user data
- A basic portfolio website

Copyright © 2025 Skill Foundry

Setting Up Your Development Environment

🚀 Install Python (Quickly)

To start web development with Python, you need Python installed on your computer. Here's how you can quickly install it:

1. **Go to the official website:**
   ➔ Open your browser and go to python.org.
2. **Download Python:**
   ➔ Click on the “Download Python” button (it will automatically show the latest version).
3. **Install Python:**
   ➔ After downloading, open the installer.
   ➔ **Important:** Make sure you **tick the checkbox** that says **"Add Python to PATH"** before clicking Install.
   (This step will save you a lot of future problems.)
4. **Verify installation:**
   ➔ Open your terminal or command prompt.
   ➔ Type:

```
python --version
```

➔ If it shows the Python version (example: Python 3.12.1), it means Python is successfully installed!

🛠 Install VS Code + Useful Extensions

To write Python code, you need a good code editor.
VS Code (Visual Studio Code) is lightweight, powerful, and perfect for beginners.

Here’s how to install:

1. **Download VS Code:**
   ➔ Go to code.visualstudio.com and download it.
2. **Install VS Code:**
   ➔ Follow the install instructions on the screen.
3. **Install Extensions:**
   Open VS Code and install these important extensions:

Copyright © 2025 Skill Foundry

- **Python Extension:**
  ➔ Search for "Python" by Microsoft and install it.
  (It adds syntax highlighting, intellisense, code linting, and more.)
- **Pylance:**
  ➔ Provides superfast, powerful Python language support.

4. **Setup Tip:**
   ➔ In VS Code, go to **Settings** → **Format On Save** and enable it.
   (It will auto-format your Python code when you save the file.)

✅ **Done!**
Now Python + VS Code are ready!

📦 Install Pip, Virtualenv (Simple Explanation)

When you build Python web apps, you'll often need to install extra libraries (like Flask, Django, etc.).
Pip and **Virtualenv** help you manage those libraries easily.

➔ What is Pip?

- **Pip** is Python's **package manager**.
- It allows you to install external Python libraries easily.

Example:

```
pip install flask
```

This command will install Flask in your system.

Good news:

- Pip is already installed when you install the latest Python versions (3.4+).
- To check if pip is installed, run:

```
pip --version
```

If you see a version, you're good to go!

➔ What is Virtualenv?

- **Virtualenv** stands for **Virtual Environment**.

Copyright © 2025 Skill Foundry

- It helps you create a **separate folder** where you can install libraries only for a specific project.
- This way, your project won't mess up your system-wide Python installation.

Example:
Imagine you have two projects:

- Project A needs Flask version 1.0
- Project B needs Flask version 2.0 Without Virtualenv, it would be a mess!

With Virtualenv, each project can have its own separate Flask version.
(No conflicts, no tension!)

🛠 How to Install Virtualenv

Install virtualenv using pip:

```
pip install virtualenv
```

That's it!
(One simple command.)

You can verify it by running:

```
virtualenv --version
```

□ Create Your First Virtual Environment

Let's now create your first virtual environment step-by-step.

1. **Open Command Prompt / Terminal**
2. **Navigate to your project folder:**

Example:

```
cd Desktop
mkdir myfirstwebapp
cd myfirstwebapp
```

3. **Create Virtual Environment:**

Command:

```
virtualenv venv
```

- venv is the name of the folder that will be created.

Copyright © 2025 Skill Foundry

- Inside venv, all your project-specific Python libraries will live.

4. **Activate Virtual Environment:**

- On Windows:

```
venv\Scripts\activate
```

- On Mac/Linux:

```
source venv/bin/activate
```

After activation:
You will see something like (venv) at the start of your terminal line.
(That means you are inside the virtual environment.)

5. **Deactivate Virtual Environment:**

When you're done, type:

```
deactivate
```

It will take you back to the system's normal Python.

✅ **Awesome!**
Now your Python development environment is 100% ready! 🛠️

Copyright © 2025 Skill Foundry

Understanding HTTP, HTML & APIs

3.1 Introduction

Before diving deep into web frameworks like Flask and Django, it is essential to understand the fundamental technologies that form the backbone of the internet — **HTTP**, **HTML**, and **APIs**.
This chapter will provide you with a clear understanding of these core concepts, enabling you to grasp how modern web applications function internally.

3.2 Understanding HTTP (HyperText Transfer Protocol)

3.2.1 What is HTTP?

HTTP is the protocol used for transferring data over the web.
Every time you open a website, your browser communicates with a server using HTTP.
It defines the rules for requests and responses between a **client** (e.g., browser) and a **server** (e.g., a web hosting server).

HTTP is a **stateless** protocol, meaning each request-response cycle is independent. Servers do not remember previous interactions unless extra mechanisms (like cookies or sessions) are used.

3.2.2 Basic Working of HTTP

- **Client initiates a request** (e.g., opening www.example.com).
- **Server processes the request** and sends back a response (e.g., the HTML page).

This interaction happens using specific HTTP methods.

3.2.3 Common HTTP Methods

Method	Purpose
GET	Retrieve information from the server.
POST	Send data to the server (e.g., form data).
PUT	Update existing data on the server.

Copyright © 2025 Skill Foundry

Method	Purpose
DELETE	Remove existing data from the server.

As a beginner web developer, you will primarily use **GET** and **POST** methods.

3.2.4 HTTP Status Codes

HTTP responses include status codes that indicate the result of the request.

Status Code	Meaning
200	OK - The request was successful.
404	Not Found - The resource was not found.
500	Internal Server Error - Server crashed.
403	Forbidden - Access is denied.

Understanding status codes is crucial when developing and debugging web applications.

3.3 Introduction to HTML (HyperText Markup Language)

3.3.1 What is HTML?

HTML is the standard markup language used to create web pages.
It structures the content on the web, telling the browser what to display and how to display it.

HTML is not a programming language; it is a markup language that defines the **structure** of your webpage.

3.3.2 Basic Structure of an HTML Document

```
<!DOCTYPE html>
<html>
<head>
    <title>Page Title</title>
</head>
<body>
    <h1>Welcome to My Website</h1>
    <p>This is a paragraph of text on my homepage.</p>
</body>
</html>
```

Copyright © 2025 Skill Foundry

Explanation:

- <!DOCTYPE html>: Declares the document type (HTML5).
- <html>: The root element of the webpage.
- <head>: Contains meta-information like the page title and links to CSS.
- <body>: Contains the visible content of the webpage.

3.3.3 Essential HTML Elements

Tag	Purpose
<h1> to <h6>	Headings (h1 is the largest, h6 is the smallest)
<p>	Paragraphs of text
<a>	Hyperlinks to other pages or websites
<img>	Embeds images
	Creates a container (division) for grouping
<form>	Captures user inputs (forms)
<button>	Creates clickable buttons

3.3.4 HTML Forms (Important for Backend Integration)

Forms are essential for collecting user input, such as login details, feedback, or file uploads.

Example of a simple form:

```
<form action="/submit" method="POST">
    <label for="name">Name:</label>
    <input type="text" id="name" name="name" required>

    <label for="email">Email:</label>
    <input type="email" id="email" name="email" required>

    <button type="submit">Submit</button>
</form>
```

Important attributes:

- action: URL where form data will be sent.

Copyright © 2025 Skill Foundry

- method: HTTP method to use (usually POST).
- required: Ensures the user must fill the field before submitting.

When integrated with frameworks like Flask or Django, the form's data will be handled by backend routes.

3.3.5 Semantic HTML

Semantic HTML uses meaningful tags to improve website accessibility and SEO.

Examples of semantic elements:

- <header>, <footer>, <article>, <section>, <nav>

Benefits:

- Makes code easier to read.
- Improves SEO (search engine optimization).
- Enhances user experience for screen readers.

3.4 Understanding APIs (Application Programming Interfaces)

3.4.1 What is an API?

An **API** is a set of defined rules and protocols that allow different software applications to communicate with each other.

In web development:

- APIs allow your web application to **send** and **receive** data from external services.
- Examples include weather services, payment gateways, social media integrations, etc.

3.4.2 How APIs Work

Imagine you are building a weather application.
Instead of building your own system to measure temperature, you simply call a **Weather API** to get real-time data.

The basic flow:

- Your app sends an **HTTP request** to the API.
- The API processes the request and sends back a **JSON** response.

Copyright © 2025 Skill Foundry

3.4.3 Introduction to JSON (JavaScript Object Notation)

APIs usually send data in **JSON** format.

JSON is a lightweight data format that is easy for both humans and machines to read.

Example of a JSON object:

```
{
  "name": "John Doe",
  "email": "john@example.com",
  "age": 30
}
```

It looks very similar to a Python dictionary.

Accessing JSON in Python is very simple:

```
import json

data = '{"name": "John", "age": 30}'
parsed_data = json.loads(data)
print(parsed_data["name"])  # Output: John
```

3.4.4 Real-World Example: Fetching Data from an API Using Python

```
import requests

response = requests.get('https://api.example.com/data')
if response.status_code == 200:
    data = response.json()
    print(data)
else:
    print("Failed to retrieve data")
```

Explanation:

- requests.get() sends an HTTP GET request.
- response.json() parses the JSON response into a Python dictionary.

3.5 Conclusion

Understanding **HTTP**, **HTML**, and **APIs** is crucial before building real-world web applications.
These foundational technologies will empower you to create functional, connected, and interactive web systems.

Copyright © 2025 Skill Foundry

As we move forward into building web apps using **Flask** and **Django**, these concepts will be used practically, reinforcing your theoretical knowledge.

Copyright © 2025 Skill Foundry

Introduction to Flask

4.1 Introduction

As a beginner in Python web development, one of the best frameworks to start with is **Flask**.
Flask is known for being **lightweight**, **flexible**, and **easy to learn**, making it perfect for building simple to moderately complex web applications quickly.

In this chapter, we will understand:

- What Flask is,
- Why it is popular among developers,
- How to set up Flask,
- How to build your very first web application.

4.2 What is Flask?

Flask is a **micro web framework** written in Python.
The term "**micro**" does not mean that Flask is limited in functionality.
Instead, it means that Flask provides the essential tools to build a web application but leaves additional features (like database integrations, form handling, authentication) to be added via **extensions**.

Key Features of Flask:

- Simple and minimalistic core.
- Highly flexible — you can structure your app as you wish.
- Wide range of extensions (Flask-SQLAlchemy, Flask-WTF, etc.).
- Built-in development server and debugger.
- RESTful request dispatching.

Flask is ideal for:

- Learning web development basics.
- Building small to medium-sized web applications.
- Creating APIs and microservices.

4.3 Installing Flask

Before we can build anything with Flask, we need to install it.

Copyright © 2025 Skill Foundry

4.3.1 Setting Up a Virtual Environment (Recommended)

It is a good practice to create a **virtual environment** for every new Python project. This isolates the project's dependencies from your system Python.

bash

CopyEdit

```
# Install virtualenv if not already installed
pip install virtualenv

# Create a virtual environment
virtualenv venv

# Activate the virtual environment
# Windows
venv\Scripts\activate
# macOS/Linux
source venv/bin/activate
```

Once activated, your terminal will show (venv) before the command prompt.

4.3.2 Installing Flask

With the virtual environment active, install Flask using pip:

bash

CopyEdit

```
pip install Flask
```

You can verify the installation by running:

bash

CopyEdit

```
python -m flask --version
```

This should display the installed Flask version.

Copyright © 2025 Skill Foundry

4.4 Creating Your First Flask App: "Hello World"

Let's build the simplest possible Flask application.

1. Create a new file called app.py.
2. Add the following code:

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def home():
    return 'Hello, World! Welcome to Flask.'

if __name__ == '__main__':
    app.run(debug=True)
```

Explanation:

- Flask(__name__) creates a new Flask web application instance.
- @app.route('/') is a **decorator** that binds the '/' URL to the home() function.
- return 'Hello, World!' sends a simple text response to the browser.
- app.run(debug=True) runs the app in **debug mode**, which automatically restarts the server when code changes are detected.

4.4.1 Running the Flask App

In the terminal, navigate to your project folder and run:

```
python app.py
```

You should see an output like:

```
Running on http://127.0.0.1:5000/
```

Open your web browser and go to **http://127.0.0.1:5000/**
You will see your "Hello, World!" message.

4.5 How Flask Handles Routes (URL Mapping)

In Flask, a **route** is a URL pattern that is linked to a Python function.
When a user accesses a particular URL, Flask executes the associated function and returns its result.

For example:

```
@app.route('/about')
```

Copyright © 2025 Skill Foundry

```
def about():
    return 'This is the About Page.'
```

Visiting **http://127.0.0.1:5000/about** will display:

```
This is the About Page.
```

4.5.1 Dynamic Routes

You can also create **dynamic routes** where parts of the URL can vary.

@app.route('/user/<username>')

def show_user_profile(username):

return f'Hello, {username}!'

- Visiting **/user/Alice** will show: Hello, Alice!
- Visiting **/user/Bob** will show: Hello, Bob!

Note:
The <username> part is a **variable** passed into the function.

4.5.2 Route Methods (GET, POST)

By default, routes answer only to **GET** requests.
To handle **POST** or multiple methods, you specify the methods parameter:

```
@app.route('/submit', methods=['GET', 'POST'])
def submit():
    if request.method == 'POST':
        return 'Form submitted successfully!'
    else:
        return 'Please submit the form.'
```

You will need to import request from Flask:

```
from flask import request
```

4.6 Summary

In this chapter, you have learned:

Copyright © 2025 Skill Foundry

- What Flask is and why it is a powerful tool for web developers.
- How to install Flask and set up your project environment.
- How to build a basic "Hello World" application.
- How URL routing works in Flask, including dynamic and method-specific routes.

These skills are foundational.
In the next chapter, we will start **building a real mini-website** using Flask, including multiple pages, templates, and forms!

Copyright © 2025 Skill Foundry

Building a Mini Website with Flask

5.1 Introduction

Now that you are comfortable with the basics of Flask, it's time to build a **multi-page mini-website**.
This project will teach you about:

- Creating multiple web pages,
- Passing data from backend to frontend,
- Using HTML templates (Jinja2),
- Handling basic forms in Flask,
- Flash messages for user feedback.

This will be your **first real-world web application** in Python.

5.2 Project Overview

We will build a small website that includes:

- Home Page (/)
- About Page (/about)
- Contact Page (/contact) with a form

We will also:

- Pass dynamic content from Flask to HTML pages,
- Handle form submissions,
- Display flash messages like "Form Submitted Successfully!".

5.3 Setting Up the Project Structure

Copyright © 2025 Skill Foundry

First, create a project folder:

```
mini_website/
│
├── app.py
├── templates/
│   ├── base.html
│   ├── home.html
│   ├── about.html
│   └── contact.html
└── static/
    ├── style.css
```

- app.py: Main application file.
- templates/: Contains all HTML templates.
- static/: Contains CSS, JavaScript, images (static files).

5.4 Creating the Flask App

In app.py, write the basic setup:

```
from flask import Flask, render_template, request, redirect,
flash, url_for

app = Flask(__name__)
app.secret_key = 'your_secret_key_here'  # Needed for flashing
messages

@app.route('/')
def home():
    return render_template('home.html')

@app.route('/about')
def about():
    return render_template('about.html')

@app.route('/contact', methods=['GET', 'POST'])
def contact():
```

Copyright © 2025 Skill Foundry

```
    if request.method == 'POST':
        name = request.form['name']
        email = request.form['email']
        message = request.form['message']

        # (In real app: Save to database, send email, etc.)

        flash('Thank you for contacting us!', 'success')
        return redirect(url_for('contact'))

    return render_template('contact.html')

if __name__ == '__main__':
    app.run(debug=True)
```

5.5 Creating HTML Templates with Jinja2

Jinja2 is the templating engine Flask uses to dynamically generate HTML.

5.5.1 base.html (Template Inheritance)

templates/base.html — This will be the common layout:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>{% block title %}My Website{% endblock %}</title>
    <link rel="stylesheet" href="{{ url_for('static',
filename='style.css') }}">
</head>
<body>

<nav>
    <a href="{{ url_for('home') }}">Home</a>
    <a href="{{ url_for('about') }}">About</a>
    <a href="{{ url_for('contact') }}">Contact</a>
</nav>

<div class="container">
    {% with messages =
get_flashed_messages(with_categories=true) %}
      {% if messages %}
        {% for category, message in messages %}
          <div class="flash {{ category }}">{{ message
}}</div>
        {% endfor %}
      {% endif %}
```

Copyright © 2025 Skill Foundry

```
    {% endwith %}

    {% block content %}{% endblock %}
</div>

</body>
</html>
```

5.5.2 home.html

templates/home.html

```
{% extends 'base.html' %}

{% block title %}Home - My Website{% endblock %}

{% block content %}
<h1>Welcome to Our Website!</h1>
<p>This is the home page.</p>
{% endblock %}
```

5.5.3 about.html

templates/about.html

```
{% extends 'base.html' %}

{% block title %}About Us{% endblock %}

{% block content %}
<h1>About Us</h1>
<p>We are passionate about building web applications using
Python and Flask.</p>
{% endblock %}
```

5.5.4 contact.html (Form Page)

templates/contact.html

```
{% extends 'base.html' %}

{% block title %}Contact Us{% endblock %}

{% block content %}
<h1>Contact Us</h1>

<form method="POST">
    <label>Name:</label><br>
```

Copyright © 2025 Skill Foundry

```
    <input type="text" name="name" required><br><br>

    <label>Email:</label><br>
    <input type="email" name="email" required><br><br>

    <label>Message:</label><br>
    <textarea name="message" required></textarea><br><br>

    <button type="submit">Send Message</button>
</form>
{% endblock %}
```

5.6 Adding Some Basic CSS

Create a file static/style.css to make the site look better:

```
body {
    font-family: Arial, sans-serif;
    margin: 0;
    padding: 0;
}

nav {
    background-color: #333;
    padding: 1em;
}

nav a {
    color: white;
    margin-right: 1em;
    text-decoration: none;
}

.container {
    padding: 2em;
}

.flash.success {
    background-color: #d4edda;
    padding: 1em;
    margin-bottom: 1em;
    border: 1px solid #c3e6cb;
}
```

5.7 Handling Form Submissions

When a user submits the form on the **Contact Page**:

Copyright © 2025 Skill Foundry

- Flask receives the data (POST request),
- You can process/store/send it (currently we just flash a message),
- The user is redirected back to the Contact page with a success notification.

Later, you can easily connect this to a database or send email notifications!

5.8 Summary

In this chapter, you have built a **real, working mini-website** with:

- Multiple pages and navigation,
- A base layout using Jinja2 templates,
- Basic form handling,
- Flash messages for user feedback,
- CSS styling for a clean frontend.

These are **fundamental skills** for any web developer, and they prepare you for **bigger projects**.

Copyright © 2025 Skill Foundry

Introduction to Django (Fast Overview)

6.1 Introduction

After mastering Flask basics, it's time to move one step ahead: **Django**.
While Flask gives you flexibility and simplicity, Django provides a **full-fledged web framework** that comes with almost everything built-in: authentication, ORM (Object-Relational Mapping), admin panels, and more.

In this chapter, we will cover:

- What Django is,
- How it differs from Flask,
- Setting up your first Django project,
- Basic understanding of Django's core components: Apps, Models, Views, and Templates.

6.2 What is Django?

Django is a **high-level Python web framework** that encourages **rapid development** and **clean, pragmatic design**.
It follows the "**batteries-included**" philosophy — meaning it comes with a lot of built-in features to handle common web development tasks.

6.2.1 Key Features of Django

- **Built-in Admin Interface**: Quickly manage your application through a web-based admin dashboard.
- **ORM (Object-Relational Mapping)**: Work with databases using Python objects.
- **Authentication System**: Built-in user management (login, logout, password management).
- **Scalability**: Django is used by big websites (Instagram, Pinterest, etc.).
- **Security**: Protects automatically against many security threats (SQL Injection, XSS, CSRF).
- **Template Engine**: Create dynamic HTML pages easily.
- **Form Handling**: Built-in forms with validation.

Copyright © 2025 Skill Foundry

6.2.2 Flask vs Django: Quick Comparison

Feature	Flask	Django
Philosophy	Microframework, minimalistic	Full-stack, batteries included
Flexibility	Very flexible	Structured and opinionated
Learning Curve	Easier for beginners	Slightly steeper
Admin Interface	Not built-in (requires setup)	Built-in
Best For	Small/Medium apps, APIs	Larger apps, complex systems
Project Structure	Free-form	Predefined project structure

In short:

- Use Flask when you want **maximum control** and a lightweight app.
- Use Django when you want **everything ready** and plan to scale big.

6.3 Installing Django

First, make sure you have a virtual environment activated.

Then install Django using pip:

```
pip install django
```

Check if Django installed correctly:

```
django-admin --version
```

You should see a version number (e.g., 5.0).

6.4 Creating Your First Django Project

Let's create a new Django project called myproject.

```
django-admin startproject myproject
cd myproject
```

You will see this folder structure:

Copyright © 2025 Skill Foundry

```
myproject/
│
├── manage.py
└── myproject/
    ├── __init__.py
    ├── settings.py
    ├── urls.py
    ├── asgi.py
    └── wsgi.py
```

6.4.1 Important Files:

- manage.py: A command-line tool to interact with your project (run server, migrations, etc.).
- settings.py: All project settings (databases, installed apps, templates, etc.).
- urls.py: URL routes configuration (which page loads at which URL).
- wsgi.py/asgi.py: Used for deployment (don't worry about them yet).

6.5 Running the Development Server

To start your Django server:

```
python manage.py runserver
```

You will see:

```
Starting development server at http://127.0.0.1:8000/
```

Open a browser and visit **http://127.0.0.1:8000/**
If everything is fine, you will see Django's welcome page:
"The install worked successfully! Congratulations!"

6.6 Understanding Django Apps

In Django, a **project** is made up of multiple **apps**.

- A **Project** is your overall website (e.g., an e-commerce platform).
- An **App** is a specific feature or module inside your website (e.g., cart system, user authentication, blog posts).

Copyright © 2025 Skill Foundry

👉 Think of a project as a house and each app as a separate room designed for a specific purpose.

6.6.1 Creating an App

Inside your project directory (myproject/), create your first app:

```
python manage.py startapp myapp
```

This will create a folder myapp/ with this structure:

```
myapp/
│
├── migrations/
├── __init__.py
├── admin.py
├── apps.py
├── models.py
├── tests.py
└── views.py
```

Brief Overview:

- models.py: Define your database tables here.
- views.py: Handle the logic for user requests and responses.
- admin.py: Register models for the Django Admin dashboard.
- migrations/: Track changes in the database schema.

6.7 Django Models (Database Layer)

Models are Python classes that define the structure of your database tables.

Example:

```
# myapp/models.py

from django.db import models

class BlogPost(models.Model):
    title = models.CharField(max_length=200)
    content = models.TextField()
    published_at = models.DateTimeField(auto_now_add=True)
```

Copyright © 2025 Skill Foundry

```
    def __str__(self):
        return self.title
```

Here:

- title is a short string.
- content is a long text.
- published_at automatically records when the post was created.

6.7.1 Applying Migrations

After defining models, you need to apply migrations to update the database:

```
python manage.py makemigrations
python manage.py migrate
```

- makemigrations: Tells Django to prepare migration files.
- migrate: Applies those migrations to the actual database.

6.8 Django Views (Handling Logic)

A **View** in Django is a Python function (or class) that takes a request and returns a response.

Example:

```
# myapp/views.py

from django.http import HttpResponse

def home(request):
    return HttpResponse("Welcome to the Home Page!")
```

This is a very basic view that just returns plain text.

Later, we will render full HTML pages using templates.

6.9 Django URLs (Routing System)

To connect your views to actual URLs, Django uses a **URL dispatcher**.

First, create a urls.py inside your app (myapp/urls.py):

Copyright © 2025 Skill Foundry

```
# myapp/urls.py

from django.urls import path
from . import views

urlpatterns = [
    path('', views.home, name='home'),
]
```

Then, link your app's URL configuration in the main project urls.py:

```
# myproject/urls.py

from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('myapp.urls')),  # Linking app urls
]
```

Now when you visit **http://127.0.0.1:8000/**, you will see:

```
Welcome to the Home Page!
```

✅ Bhai yahan tak aaya hai:

- You created an app.
- You learned what models, views, and URLs are.
- You made a simple functional homepage.

6.10 Django Templates (Frontend Layer)

Instead of returning plain text, we usually want to show full HTML pages.

Let's render an HTML template.

6.10.1 Create a Templates Folder

Inside your app (myapp/), create:

```
myapp/
└── templates/
    └── home.html
```

Copyright © 2025 Skill Foundry

In home.html, add:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Home Page</title>
</head>
<body>
    <h1>Welcome to My Django Website!</h1>
    <p>This page is rendered using Django Templates.</p>
</body>
</html>
```

6.10.2 Modify the View to Render Template

Update myapp/views.py:

```
from django.shortcuts import render

def home(request):
    return render(request, 'home.html')
```

Now when you refresh **http://127.0.0.1:8000/**, Django will render your beautiful HTML page!

6.11 Django Admin Panel

Django comes with a powerful **Admin Interface** to manage your app's content easily, such as adding, updating, or deleting blog posts, users, etc.

6.11.1 Enabling Admin Panel

In myapp/admin.py, register your model:

```
from django.contrib import admin
from .models import BlogPost

admin.site.register(BlogPost)
```

Now you can access the Django Admin Panel by visiting **http://127.0.0.1:8000/admin/**, but you need to create a superuser first:

Copyright © 2025 Skill Foundry

```
python manage.py createsuperuser
```

Follow the instructions to set up a username and password.

6.12 Quick CRUD Operations

In real applications, you'll want to allow users to **Create, Read, Update, and Delete** data. Django provides simple tools for these actions.

6.12.1 Example for CRUD (BlogPost)

Create a Blog Post (via Admin Panel)

- Log in to the Django Admin and create a blog post using the form.

Read (Display Blog Posts)

In your views.py:

```
from django.shortcuts import render
from .models import BlogPost

def blog_list(request):
    posts = BlogPost.objects.all()
    return render(request, 'blog_list.html', {'posts': posts})
```

Now you can display all blog posts in a template:

```
{% for post in posts %}
    <h2>{{ post.title }}</h2>
    <p>{{ post.content }}</p>
{% endfor %}
```

6.13 Conclusion of Django Basics

In this chapter, we:

- Installed Django and set up a project.
- Created an app and understood models, views, and templates.
- Learned how to use Django's built-in Admin panel.

Copyright © 2025 Skill Foundry

- Did a quick overview of CRUD operations.

Copyright © 2025 Skill Foundry

Creating a Simple Blog App with Django

7.1 Introduction to the Blog App

In this chapter, we'll create a simple **Blog** application using Django. The blog will have:

- A **model** for storing blog posts.
- Views to display the posts.
- A basic **admin panel** to manage posts.
- A simple **template** to display the blog.

7.2 Setting Up the Blog App

7.2.1 Create the Blog App

Navigate to your project directory and create the blog app:

```
python manage.py startapp blog
```

You'll get a new folder structure:

```
blog/
│
├── migrations/
├── __init__.py
├── admin.py
├── apps.py
├── models.py
├── views.py
└── urls.py
```

7.2.2 Register the Blog App in Settings

In myproject/settings.py, add 'blog' to the INSTALLED_APPS list:

```
INSTALLED_APPS = [
    # other apps
    'blog',
]
```

Copyright © 2025 Skill Foundry

7.3 Blog Post Model

We will create a simple BlogPost model to store each blog post's **title**, **content**, and **publish date**.

Open blog/models.py:

```
from django.db import models

class BlogPost(models.Model):
    title = models.CharField(max_length=200)
    content = models.TextField()
    published_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.title
```

7.3.1 Migrate the Model

After creating the model, run the following commands to apply the changes:

```
python manage.py makemigrations blog
python manage.py migrate
```

This will create the necessary database tables for your BlogPost model.

7.4 Displaying Blog Posts (Views and Templates)

7.4.1 Create the View to Display Posts

In blog/views.py, define a view to fetch all blog posts and display them:

```
from django.shortcuts import render
from .models import BlogPost

def blog_list(request):
    posts = BlogPost.objects.all()  # Fetch all blog posts
from the database
    return render(request, 'blog/blog_list.html', {'posts':
posts})
```

7.4.2 Create the Template for Blog List

Copyright © 2025 Skill Foundry

Inside your app's templates folder, create a sub-folder named blog/ and add a template file blog_list.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Blog List</title>
</head>
<body>
    <h1>Blog Posts</h1>
    <ul>
        {% for post in posts %}
            <li>
                <h2>{{ post.title }}</h2>
                <p>{{ post.content }}</p>
                <small>Published on: {{ post.published_at
}}</small>
            </li>
        {% endfor %}
    </ul>
</body>
</html>
```

7.4.3 Add URL Route for Blog List

In blog/urls.py, set up the URL to access the blog list view:

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.blog_list, name='blog_list'),
]
```

Then, in myproject/urls.py, include the blog app's URLs:

```
from django.urls import path, include

urlpatterns = [
    path('blog/', include('blog.urls')),
]
```

7.5 Admin Panel for Blog Posts

Copyright © 2025 Skill Foundry

Now, let's make sure we can manage blog posts through the Django Admin interface.

7.5.1 Register BlogPost in Admin

In blog/admin.py, add the following code to register the BlogPost model:

```
from django.contrib import admin
from .models import BlogPost

admin.site.register(BlogPost)
```

7.6 Viewing and Managing Blog Posts

Now, go to **http://127.0.0.1:8000/admin/**.
You'll need to log in with the superuser account that you created earlier using python manage.py createsuperuser.

Once logged in, you'll see the **BlogPost** model listed.
Click on it to add, edit, or delete blog posts directly from the admin panel.

7.7 Conclusion of Blog App

In this chapter, you have:

- Created a **BlogPost model** to store blog posts.
- Created a **view** to display all blog posts.
- Built a **template** to show blog posts in a user-friendly way.
- Registered the model in the **admin panel** to manage posts easily.

This is a basic blog app. You can extend it with more features like:

- Pagination for many posts.
- Edit/Delete functionality in the frontend.
- User authentication for managing posts.

Copyright © 2025 Skill Foundry

Hosting Your Web App

8.1 Introduction to Hosting

Hosting means making your website or web app accessible to everyone on the internet.

Locally, your app runs at http://127.0.0.1:8000/, but hosting makes it available publicly, like https://mywebsite.com/.

8.2 Choosing a Hosting Platform

For beginners, the easiest option is **PythonAnywhere**.
It's free (for small apps) and made especially for Python web apps like Flask and Django.

Other options (for later when you grow bigger):

- Heroku
- Render
- DigitalOcean (for more control)

But for now, let's use **PythonAnywhere**.

8.3 Deploying a Flask App to PythonAnywhere (Quick Steps)

Step 1: Prepare Your Flask App

Make sure your Flask app looks like this (for example):

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def home():
    return "Hello, World!"
```

Save it in a file named app.py.

Step 2: Create an Account on PythonAnywhere

- Go to pythonanywhere.com
- Sign up for a free account.

Copyright © 2025 Skill Foundry

Step 3: Upload Your Project

- In PythonAnywhere Dashboard, click on **Files** > **Upload a File**.
- Upload your app.py file.

If your app has other files (templates, static files), you should upload them too, keeping the same folder structure.

Step 4: Set Up a Web App

- Click **Web** > **Add a New Web App**.
- Choose **Flask** > Choose Python version (same as your local).
- Set your app's working directory correctly.
- In the **WSGI file** (e.g., /var/www/yourusername_pythonanywhere_com_wsgi.py), make sure it points to your app object inside app.py.

Example WSGI content:

```
import sys
path = '/home/yourusername/path_to_your_project'
if path not in sys.path:
    sys.path.append(path)
```

from app import app as application

(Here, application is required by PythonAnywhere.)

Step 5: Reload and Visit Your Site

- After setup, click the **Reload** button.
- Visit the provided link (something like https://yourusername.pythonanywhere.com/)
  Boom! 🎉 Your Flask app is live!

Copyright © 2025 Skill Foundry

8.4 Deploying a Django App to PythonAnywhere (Quick Steps)

Deploying Django is slightly bigger than Flask, but still simple.

Step 1: Prepare Your Django Project

Make sure:

- **ALLOWED_HOSTS** in settings.py is set correctly:

```
ALLOWED_HOSTS = ['yourusername.pythonanywhere.com']
```

- Static files are properly collected:

```
python manage.py collectstatic
```

(This command gathers all static files in one place for production.)

Step 2: Upload Your Django Project

Same as Flask:

- Upload your full project folder to PythonAnywhere.

Step 3: Set Up a Virtual Environment (Recommended)

- In the **Console** tab, create and activate a virtual environment:

```
python3 -m venv myenv
source myenv/bin/activate
```

- Install Django:

```
pip install django
```

(Or install other requirements if you have a requirements.txt file.)

Step 4: Set Up the Web App

- Go to **Web** > **Add New Web App** > Manual Configuration > Python.
- Set the path to your project and WSGI file.

Modify the WSGI file (/var/www/yourusername_pythonanywhere_com_wsgi.py) to point to your Django app.

Example:

```
import sys
import os
```

Copyright © 2025 Skill Foundry

```
path = '/home/yourusername/path_to_your_project'
if path not in sys.path:
    sys.path.append(path)

os.environ['DJANGO_SETTINGS_MODULE'] = 'myproject.settings'

from django.core.wsgi import get_wsgi_application
application = get_wsgi_application()
```

Step 5: Run Migrations & Collectstatic

In the PythonAnywhere Bash console:

```
python manage.py migrate
python manage.py collectstatic
```

Step 6: Reload and Go Live

- Click **Reload** on the Web tab.
- Visit your live site at https://yourusername.pythonanywhere.com/.

Congratulations! □ Your Django app is now hosted!

8.5 Real-World Hosting Tips

- Always use a virtual environment.
- Set DEBUG = False in settings.py for production.
- Use strong passwords for Django Admin.
- Regularly back up your database.
- Upgrade to paid plans if you need more resources.

Copyright © 2025 Skill Foundry

Bonus Projects (Mini Real World Projects)

9.1 Introduction to Bonus Projects

Projects are the **fastest way to master web development**.
Yahan hum kuch simple but impactful projects banayenge jo:

- Teri Django ya Flask skills polish karenge
- Tujhe portfolio ke liye projects denge
- Confidence build karenge real-world apps banane ka

9.2 Project 1: Personal Portfolio Website (Flask or Django)

Goal:

A website to showcase your skills, projects, and contact details.

Features:

- Home Page (intro about yourself)
- Projects Page (list of projects with links)
- Contact Page (simple form submission)

How to Do:

- Flask: Create 3 routes (/, /projects, /contact)
- Django: Create a simple app with 3 templates.

Extra Polish:

- Add Bootstrap to make it responsive.
- Form submission can save messages (optional).

9.3 Project 2: To-Do List Web App (Flask or Django)

Goal:

A simple app where users can add and complete tasks.

Features:

- Add a new task
- Mark task as complete

Copyright © 2025 Skill Foundry

- Delete task
- View all tasks

Database Design (Very Basic):

Field	Type
title	CharField
is_completed	BooleanField

How to Do:

- Use forms to add tasks.
- Use buttons to mark as complete or delete.

9.4 Project 3: Simple Notes App (Flask or Django)

Goal:

An app where users can write, edit, and delete notes.

Features:

- Add a new note (Title + Content)
- Edit existing notes
- Delete notes
- List of all notes

Database Design:

Field	Type
title	CharField
content	TextField
created_at	DateTimeField

How to Do:

- Basic CRUD operations (Create, Read, Update, Delete).
- Use Django's Admin Panel or Flask Forms.

9.5 Project 4: Blog CMS (Content Management System using Django)

Copyright © 2025 Skill Foundry

Goal:

Create a mini **Blog Management System** where admin can:

- Add blog posts
- Edit blog posts
- Delete blog posts
- Show posts on the public website

This will feel like a mini WordPress!

Features:

- Django Admin panel to manage posts.
- Frontend to show blog posts.
- URL structure like /blog/slug-title.

Database Design (Model):

Field	Type
title	CharField
slug	SlugField (for URL)
content	TextField
created_at	DateTimeField

Advanced Touch (Optional):

- Add categories (Tech, Lifestyle, etc.)
- Add tags to posts
- Add a simple search feature

9.6 Why These Projects?

✅ They are small enough to finish in **1-2 days** each.
✅ They teach you **core skills**: routes, forms, models, templates.
✅ You can proudly show them in your portfolio or GitHub.

Copyright © 2025 Skill Foundry